Project Contribution Report

Name: Afnan Abdulkadir
University: Aston University
Course: Summer School in Robotics

1. Introduction

This report documents my individual contribution to our robotics project, which I worked on with my teammate Juan Manuel. It involved developing a robot navigation system in ROS 2 (Jazzy). The task was for the robot to navigate a maze and reach the goal using the shortest possible path. We utilised Aruco markers for localisation and applied a grid-based path planning approach, converting pixel coordinates into real-world measurements.

The objective was to enable the robot to determine its position within a mapped environment, plan an optimal route to the target location, and execute smooth movement along that path in real-world coordinates.

This report outlines my specific tasks, highlights successful implementations, identifies areas for improvement, and provides feedback on the course content and execution. Code samples and technical details are included in the appendix where relevant.

2. How have I contributed to the project work?

My teammate and I collaborated closely throughout the project, evenly dividing tasks while supporting each other whenever needed. The early stages were focused mainly on planning — deciding on our overall approach, selecting the algorithm for determining the shortest path, and outlining how the different components would interact. I took a proactive role in these early phases, carrying out extensive research from multiple sources to help us define our strategy.

Our development pipeline followed a clear sequence which I've shortened down to 4 phases:

1. **Grid Creation** – Generated an occupancy map of the maze by detecting ArUco markers and mapping their positions into a 2.5 m × 2.5 m virtual grid (0.25 m cell size).

2. **Marker Mapping** – Subscribed to the /aruco/markers/transformed topic (aruco_msgs/ArucoMarkers) to read marker IDs and positions, overlaying them on the occupancy grid image.

3. **Pathfinding** – Computed the shortest path from start to goal using a custom A* implementation, outputting a sequence of grid coordinates.

4. **Movement Execution** – Translated the optimal path into movement commands for the robot.

My key contributions within this pipeline were:

- **A* Pathfinding Implementation** – Developed the ROS 2 node that implements the A* algorithm to generate the optimal path. I researched various pathfinding methods, and we chose A* for its balance between accuracy and efficiency. I handled its full

implementation and integration. *(see figure 1.1 and figure 1.2 in appendix)*

- **Grid Generation from Markers** – Created the script (Figure2) to read Aruco marker data through the topic '/aruco/markers/transformed' and translated it into a visual grid (PNG format). This served as the foundation for the other components in our system, which we later built upon.

- **Rotation and Heading Control** – Designed logic (figure 3) to adjust the robot's heading so it would face the correct direction before moving to the next waypoint. Sometimes the robot wouldn't be facing the right direction to start the path and therefore would do nothing, so I had to implement this logic to help it be facing the right direction in case it wasn't.
- **Debugging and Testing** – Diagnosed and resolved issues such as endless rotation near the goal, marker misalignment, and mapping inaccuracies. I repeatedly tested the robot to verify stable behaviour.

- **Documentation** – Produced detailed internal notes covering setup and testing to make it easier for me and my teammate to troubleshoot. Screenshots of these notes are included in the appendix. *(Figure 4)*

3. What I did that worked well

I am most proud, first and foremost, of the effort I put into this project. At the beginning, I had very little knowledge of ROS 2 and often felt confused during the initial stages. However, I committed significant time to learning the material and conducting independent research. On some days, I stayed in the lab until 10 p.m., continuously testing and optimising the robot's behaviour.

Coarse Path Optimisation – I refined the navigation path so that the robot would move in increments of one 0.25 m grid cell, reducing unnecessary micro-movements and improving efficiency. Originally, the grid was too fine and disorganised and typical maze runs averaged around 80 steps (sometimes even higher); after implementing the coarser grid, this was reduced to roughly 20 steps, cutting traversal length immensely. The larger grid cells also simplified movement control and significantly reduced accumulated heading error over the course of a run.

Robust Pose Handling – In later stages, I improved localisation reliability by filtering ArUco detections to include only the required marker ID and ignoring noisy or irrelevant detections (like other robots that were physically on the maze). *(see figure 5).*

Clear Debug Logging – I implemented targeted logging messages (e.g., "Goal reached" and "Getting closer to the goal") within the movement control node. These logs provided real-time insight into state transitions, enabling faster diagnosis of logic errors. This also helped my teammate stay fully informed during tests, allowing us to collaboratively troubleshoot without having to read through raw ROS topic output. *(See Figure 6 in the appendix.)*

Extensive Testing & Micro Adjustments – I carried out multiple test runs to address rotation accuracy, which was a recurring issue in early trials. Initially, the robot overshot 90° turns by roughly 10–15°, leading to small but consistent path deviations. By gradually tuning the control parameters — particularly reducing angular velocity and experimenting with different rotation speeds — I was able to achieve more controlled turns and eliminate the overshoot problem.

These improvements directly contributed to a smoother and more predictable robot movement.

4. What our team did that worked well

As a team, we excelled in several key areas:

Communication – We regularly mind-mapped our ideas and took time to reflect on our methodology whenever the code was not functioning as expected. This allowed us to adapt quickly and find workable solutions. Clear code structure and consistent formatting across our files meant that we could easily read and understand each other's work, which made integration much smoother.

Integrating Multiple Modules – We successfully combined mapping, path planning, and robot control scripts into one cohesive workflow, following the pipeline I described earlier on.This modular approach made the system easier to debug and maintain.

Early Simulation Setup – We began early by creating a simulation test environment using virtual grids. This allowed us to validate key logic before deploying on the physical robot, reducing integration problems later.

A* Pathfinding – We collectively decided on and implemented an A* algorithm that reliably found optimal paths through the occupancy map while avoiding walls and obstacles.

Map Visualisation – We built a visual occupancy grid that displayed real-time Aruco marker overlays along with maze walls and obstacles (shown in red). This provided immediate visual feedback on both robot position and environment layout.

These successes were driven by effective task division, early testing, regular integration checks, and strong communication throughout the project.

5. What could I have done better?

A major issue arose from a misunderstanding of our sensor data. We initially believed we were reading values from a gyro sensor, but in reality, we were using yaw angles derived from the robot's wheel odometry via the `/odom` topic. This data was not from an actual gyro and was therefore less reliable, especially for precise rotation control. Unfortunately, we discovered this quite late in the project, which meant we had already built large parts of our system around incorrect assumptions.

At first, we had planned to use AR tag detections to dynamically recalibrate the robot's position during movement. However, the AR tag data from the ROS topic proved highly unstable, often causing unexpected behaviour. This forced us to fall back on the yaw angles from odometry. When that still produced inconsistent results, and it was too late to implement a real gyro sensor, the only solution was to adopt a fixed-path approach as a workaround.This fixed-path method took the initial list of path coordinates, saved them into a `.csv` file, and then executed simple movement and turning commands. While easier to implement, it left very little room for optimisation or adaptability in the robot's navigation. It meant we had to rely on fixed thresholds instead of automating tolerance adjustments for heading and position. Using real-world coordinates directly from AR tags (with better filtering and stabilisation) could have allowed for smoother, more accurate movement.

In conclusion, if we were to redo the project, we would prioritise implementing a true gyro sensor early on to ensure accurate and reliable rotation control from the start.

6. Course Feedback

6.1 Liked Content

I really enjoyed the hands-on use of ROS 2 and real-world robotics concepts, which gave me a lot of practical experience. I also appreciated the opportunity to work with computer vision through ArUco marker detection. Much of my learning came from trial and error, which made the process engaging and rewarding.

Having to learn so many new skills like: integrating mapping, path planning, and control into one cohesive project was a highlight, and it also demonstrated how different components of a robotics system fit together. I also valued how collaborative and team-focused the course was — we constantly worked in groups or pairs, which allowed me to learn from others, gain different perspectives, and work with a diverse group of people. The instructors were very helpful and supportive throughout the course.

6.2 Execution i liked:

I also liked having full access to physical robots for testing, as it allowed us to experiment freely. I also made good use of the instructors' availability during lab sessions for real-time troubleshooting, which was extremely helpful.

6.3 Disliked Content

I would have liked to see more theory-based sessions, particularly at the start of the course. Coming in with no prior knowledge of ROS meant that some concepts were difficult to grasp initially, and more theory early on could have helped build a stronger foundation.

6.4: Execution I didn't like:

Furthermore the occasional hardware availability bottlenecks slowed progress. There were also times when our ArUco markers wouldn't be detected due to external factors outside our control, such as room lighting or camera traffic on the network.

6.5 Suggestions

Though I enjoyed the overall layout of the course and the practical sense of it, my suggestions for improvement would be to incorporate more structured debugging exercises to better prepare students for real-world integration challenges. I would also recommend including more theory-based content — not only at the start of the course but revisited regularly throughout the term — to reinforce key concepts and keep students' knowledge fresh. Finally, increasing the number of intermediate peer review sessions would provide more opportunities for teams to exchange feedback, identify issues early, and improve the overall quality of their projects.

See below for appendix

Appendix:

```python
def a_star(image, start_px, goal_px, cell_size):
    height, width, _ = image.shape
    grid_w = width // cell_size
    grid_h = height // cell_size

    def heuristic(a, b):
        return abs(a[0] - b[0]) + abs(a[1] - b[1])  # Manhattan distance

    def neighbors(cell):
        x, y = cell
        for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
            nx, ny = x + dx, y + dy
            if 0 <= nx < grid_w and 0 <= ny < grid_h:
                world_x, world_y = nx * cell_size, ny * cell_size
                if is_cell_black(image, world_x, world_y, cell_size, ignore_pos=start_px):
                    yield (nx, ny)

    start = (start_px[0] // cell_size, start_px[1] // cell_size)
    goal = (goal_px[0] // cell_size, goal_px[1] // cell_size)

    frontier = PriorityQueue()
    frontier.put((0, start))
    came_from = {start: None}
    cost_so_far = {start: 0}

    while not frontier.empty():
        _, current = frontier.get()
        if current == goal:
            break
        for next_cell in neighbors(current):
            new_cost = cost_so_far[current] + 1
            if next_cell not in cost_so_far or new_cost < cost_so_far[next_cell]:
                cost_so_far[next_cell] = new_cost
                priority = new_cost + heuristic(goal, next_cell)
                frontier.put((priority, next_cell))
                came_from[next_cell] = current

    # Reconstruct path
    current = goal
    path = []
    while current and current != start:
        cx, cy = current
        path.append(get_cell_center(cx * cell_size, cy * cell_size, cell_size))
        current = came_from.get(current)
    path.reverse()
```

Figure 1.1: My A* search implementation over a grid-based occupancy image. Walkable areas are inferred based on pixel color. This is a snippet of the whole code.

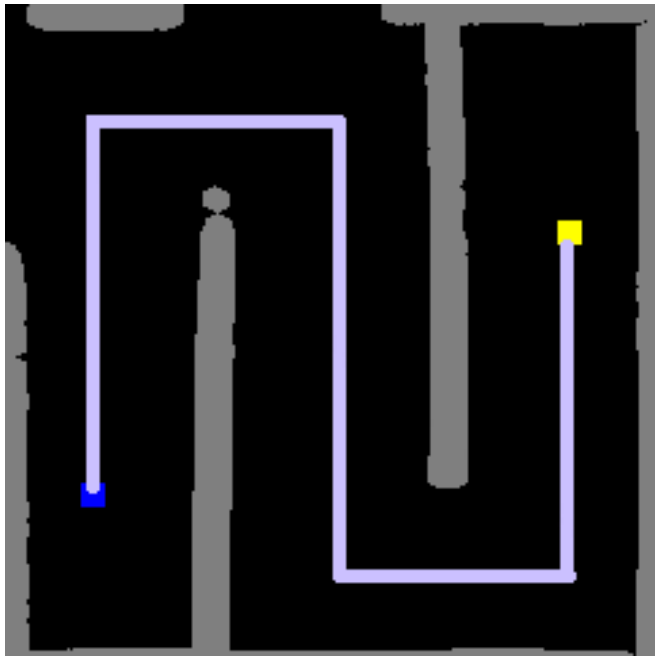

Figure 1.2: An example of the image generated by the A* algorithm file. The blue represents our robot (ID 10) and yellow represents the goal.

```
image = np.zeros((height, width), dtype=np.uint8)
image[data == 0] = 255       # Free space = white
image[data > 0] = 0          # Occupied = black
image[data < 0] = 127        # Unknown = gray

image_rgb = np.stack([image] * 3, axis=2)

# Draw markers here (simplified)
for marker in self.latest_markers:
    pos = marker.pose.position
    px, py = world_to_pixel(pos.x, pos.y)
    color = (255, 255, 0) if marker.id == 15 else (0, 0, 255)
    image_rgb[py-2:py+3, px-2:px+3] = color  # Small square

Image.fromarray(image_rgb).save("/home/pi/ros2_ws/occupancy_map_with_markers.png")
self.get_logger().info(" ✅ Saved map with markers to: ~/ros2_ws/occupancy_map_with_markers.png")
```

Figure 2: This snippet creates of my virtual grid file generates image using ROS occupancy data and marks the robot and goal positions based on incoming ArUco marker data

```python
angle_to_target = math.atan2(dy, dx)  # Desired heading
heading_error = angle_to_target - self.current_heading
heading_error = (heading_error + math.pi) % (2 * math.pi) - math.pi  # Normalize to [-π, π]

# Decide whether to rotate or move forward
if abs(heading_error) > 0.1:  # Threshold: ~5.7 degrees
    # Rotate to face the target
    twist = Twist()
    twist.angular.z = 0.5 if heading_error > 0 else -0.5
    twist.linear.x = 0.0
    self.publisher.publish(twist)
    self.get_logger().info(f"🔄 Rotating to face target, heading error: {math.degrees(heading_error):.2f}°")
    return
else:
    # Move forward
    twist = Twist()
    twist.linear.x = 0.1
    twist.angular.z = 0.0
    self.publisher.publish(twist)
    self.get_logger().info("🚀 Heading aligned. Moving forward.")
```

Figure 3: This snippet computes the heading angle to the next waypoint and compares it with the robot's current heading. If the heading error exceeds a threshold, the robot rotates in place before moving forward, ensuring proper alignment at each step.

Important commands:

- To build: `colcon build`
- To source : `source install/setup.bash`
- To launch/run the file use this command: `ros2 launch/run (nameofpackage) (nameoffile)`
- To list out your packages: `ros2 pkg list`
- To see which nodes are running at the moment: `ros2 node list`
- To see all the topics: `ros2 topic list`
- To see a specific topics output: `ros2 topic echo /topicname`

commonly ran commands:

ros2 run aruco_ros2 go_position1

sudo apt update

sudo apt install ros-jazzy-cv-bridge

cd ~/ros2_ws

colcon build --packages-select aruco_interfaces

source install/setup.bash

colcon build --packages-select aruco_ros2

source install/setup.bash

id: 10

pose:

position:

x: -1.7133246108505056

y: -0.4355338591808478

z: 0.37894713462885665

orientation:

x: -0.05990409349191522

Figure 4: A snippet of the internal notes I created during development, documenting key ROS 2 commands, ArUco troubleshooting tips etc. These notes helped streamline setup and were especially useful during debugging and testing.

```python
def markers_callback(self, msg: ArucoMarkers):
    for marker in msg.markers:
        if marker.id == 10:  # Filter for robot's marker only
            self.robot_pose = marker.pose
            self.received_pose = True
            self.get_logger().info(f"🤖 Found robot marker at ({marker.pose.position.x:.2f}, {marker.pose.position.y:.2f})")
            break
```

Figure 5: The node filters ArUco detections to include only the robot's specific marker ID (e.g., ID 10), improving robustness by ignoring irrelevant detections such as other robots on the maze.

```python
def follow_path(self):
    if self.current_index >= len(self.path):
        self.stop_robot()
        self.get_logger().info("✅ Goal reached. Path complete.")
        self.timer.cancel()
        rclpy.shutdown()
        return

    # Get current target waypoint
    target_pixel = self.path[self.current_index]
    target_x, target_y = self.pixel_to_meter(target_pixel)

    # Calculate vector to target
    dx = target_x - self.current_pos[0]
    dy = target_y - self.current_pos[1]
    distance = math.hypot(dx, dy)

    # 🔍 LOG: Where we're going, where we are, how far
    self.get_logger().info(f"➡️ Moving to point {self.current_index + 1}/{len(self.path)}: "
                           f"Target=({target_x:.2f}, {target_y:.2f}), "
                           f"Current=({self.current_pos[0]:.2f}, {self.current_pos[1]:.2f}), "
                           f"Distance={distance:.3f} m")

    # 🔍 LOG: Check if we're getting closer or not
    if self.previous_distance is not None:
        if distance < self.previous_distance:
            self.get_logger().info("📉 Getting closer to the goal.")
        else:
            self.get_logger().warn("📈 Getting farther from the goal!")
```

Figure 6: As shown, I implemented clear logging to show the robot's target, current position, distance, and whether it's getting closer or farther — which helped a lot during testing and debugging. These are only some of the examples of logging I used for debugging.